



ROBOCUPJUNIOR RESCUE (LINE/MAZE) 2026

TEAM DESCRIPTION PAPER AURAT

Abstract

This paper presents the autonomous rescue robot developed by team AURAT for the RoboCupJunior Rescue Maze 2026 World Finals. Designed to achieve full exploration and precise victim identification within a strict 8-minute time limit, the robot features a robust, modular three-level physical architecture built with a custom vertical suspension system and hollow-rimmed TPU tires to safely navigate speedbumps and 25° ramps. The hardware ecosystem coordinates an Arduino Mega for low-level motion control, a custom 4-layer PCB for noise-free electronics routing, and a Raspberry Pi 5 acting as the high-level master node. To optimize computational efficiency, the software implements a concurrent, multi-threaded architecture utilizing Dijkstra's algorithm for real-time dynamic pathfinding. Following technical challenges encountered during the national qualifiers in Catania, the vision pipeline was successfully upgraded from a blocking full-frame YOLOv8 model into an optimized, two-tiered framework. Spatial-chromatic blob localization is offloaded to an OpenMV H7 Plus edge camera, while character classification is executed on the Raspberry Pi 5 via a high-speed, rotation-invariant K-Nearest Neighbors (KNN) pipeline. System integration testing validates that these concurrent refinements guarantee deterministic obstacle navigation, robust cell alignment, and highly resilient target identification under variable arena conditions

1. Introduction

a. Team

Berra Loris: Electronics student at ISIS C. Facchinetti, participated in robocup for 3 years in 3 different categories: home playground, onstage advanced and rescue maze. Experience working at *STMicroelectronics* developing projects like an automated greenhouse and at *Pomini* long rolling mills. Participated at "JA impresa in azione" building a simulated startup studying the economics needed to start a company in which we developed a sensor system for heavy vehicles to reduce incidents in cities, the system uses ultrasound sensors and AI recognition with yolo trough cameras. Worked on the electronics side of the project including management of the sensors and development of the PCB.

Marco Moretti: Computer Science student at ISIS C. Facchinetti, Lead Software Engineer, and Python and AI Specialist, responsible for high-level mapping, pathfinding, and computer vision. Participated in RoboCup Junior 2025 in the Italian Rescue Maze Championship (10th place) and in the Onstage Advanced World Cup in Salvador (awarded best technical documentation and classified among the top 5 teams), followed by a 5th-place finish in the 2026 Italian Rescue Maze Championship.

Sandra D'Arrigo: Computer Science student at ISIS C. Facchinetti and Arduino specialist. Participated in RoboCup Junior 2025 in the Italian Rescue Maze Championship (10th place), assisted in creating the costumes and the poster for the 2025 World Championship in Brazil, and also participated in the 2026 edition of the same competition, achieving a 5th-place placement.

Federico Barban: Computer Science student at ISIS C. Facchinetti, 3D Modelist, and Mechanical Specialist. Active in 3D modeling since the age of 12, responsible for modeling all the robot's components from scratch. This marks the very first participation in a robotics competition

2. Project Planning

a. Overall Project Plan

Our primary objective for the 2026 World Finals is to place within the top twelve teams, a realistic yet ambitious goal considering that this is our first appearance in this category at the world level. To achieve this, our focus is on completing a full and reliable exploration of the maze, correctly identifying victims using the detection

algorithms and technologies described in the following section, and respecting constraints such as colored tiles and checkpoints. We also aim to consistently secure the exit bonus, which proved decisive during the nationals competitions. Another key objective is to successfully traverse the Dangerous Zone in order to further refine the strategies and mechanical solutions we developed throughout this year. To meet the competition's constraints, our robot was designed to comply with all physical, environmental, and operational requirements defined in the rulebook.

- **Time constraints (8 minutes).** To efficiently operate within the time limit, the robot maintains an internal map that tracks explored areas, previously visited tiles, and points of interest. This allows the navigation system to avoid redundant paths and optimize exploration time.
- **Physical constraints:** the robot needs to have a maximum height of 25 cm and we were able to achieve this thanks to a compact and modular design. For optimal movements and stability the robot needed to keep an optimal center of mass, while using advanced sensors and voluminous components (like the PCB, the motors, the batteries, etc.). The rescue kits needed to be designed to respect the dimensions constraints (at least 1cm³), and so we also needed to design a release system compliant with the constraints while keeping great performances.
- **Environmental constraints:** the maze is full of bumps (1 to 2 cm depending on the zone), ramps, stairs, and many other types of obstacles. So, we developed a custom suspension system along with custom designed and 3d printed tires in order to make the robot able to easily handle everything on its way.

Sensor Coordination for stop conditions

The robot must correctly handle mandatory stop conditions imposed by the field, such as blue tiles, checkpoints, and black tiles. To achieve this, we coordinate multiple sensors:

- Color sensor to detect blue, black, and checkpoint tiles
- Time of Flight sensor to confirm tiles boundaries and ensure the robot is fully inside the cell
- IMU to maintain orientation during the stop
- Internal timers to enforce the required waiting period

When the robot enters a blue tile, the software triggers a mandatory 5 second stop. The firmware freezes locomotion, monitors the timer, and only resumes movement once the full duration has elapsed, preventing a Lack of Progress penalty. A similar mechanism is used for checkpoints and black tiles, each with its own logic and safety condition

We started designing the entire robot body from scratch, beginning with a simple base plate and four vertical columns for the upper structure. The initial design also included four motor mounting rails. These rails represented our first design mistake and were later replaced with two guide rods for each motor to improve mechanical stability and alignment. We carried out extensive tests on wheel development, and after a printing error we realized that the wheel rims needed to be hollow to reduce weight and improve performance. To connect the motors to the rest of the robot, we designed a motor housing with a T-shaped protrusion allowing vertical movement, supported by two inclined shock absorbers for each housing. However, this configuration proved to be unstable. As a result, we redesigned the system using vertical shock absorbers, which significantly improved stability.

Hardware testing began in September. To resolve I2C address conflicts among the ToF sensors, an initial multiplexer approach was replaced by utilizing the sensors' shutdown pins to dynamically reassign unique addresses at boot. After validation on breadboards, development shifted to the first prototype PCB. For status signaling, an initial 5mm PCB LED proved too dim and was replaced with a 12V LED strip, integrating a MOSFET to interface the high-power component with the low-voltage control logic.

A power issue with the rescue kit deployment servo initially caused current spikes, triggering the Arduino's protection and resetting the microcontroller. After evaluating alternative powering methods, a dedicated 7805 voltage regulator was integrated to supply stable 5V and adequate current.

The software architecture evolved from a single-process Python script, incapable of managing concurrent real-time tasks like LoPs and black tile detection, into a multi-threaded system running parallel operations (see Section 4.a). Furthermore, during the national competition in Catania, the initial YOLOv8 vision model on the Pi CPU proved too slow and highly sensitive to lighting conditions. To enable robust, real-time victim recognition

anywhere within a tile, the architecture was upgraded by offloading video processing to a dedicated edge camera and replacing YOLO with a fast KNN classifier.

Early trials revealed a constant drift in straight-line motion, which was corrected by implementing an IMU-based PD controller, alongside a full PID controller for precise 90-degree turns. Additionally, during the competition in Catania, a cumulative rotation error was identified, caused by adding relative target angles to the current orientation. This was resolved by modifying the firmware to reference a fixed global orientation. To eliminate any remaining offset derived from a crooked initial placement, the current system utilizes two lateral ToF sensors per side to dynamically calculate the wall-skew angle, ensuring the robot constantly straightens and centers itself within the cell.

As the system grew, we also redesigned the communication layer: the initial long messages caused delays, so we developed a compact protocol using a single character plus a numeric parameter. This made the exchange between Raspberry Pi and Arduino much faster and more reliable. Once all sensors were integrated, we added the logic for handling special tiles. Using the color sensor and ToF readings, the firmware detects blue tiles, checkpoints, and black tiles, enforces the mandatory stop, and resumes movement only when the timer expires, preventing LoPs. Finally, to comply with the rulebook, we added a “back-to-front” mode: instead of duplicating code, a simple inversion flag remaps sensors and reverses motor logic while keeping all algorithms unchanged. In the last iteration we focused on robustness, adding safety limits, filtering noisy data, and improving error handling. These refinements made the firmware stable, responsive, and fully integrated with the high-level Python software.

b. Integration Plan

Data journey starts from the sensors (ToF, gyroscope, encoder, colour and cameras) then through different methods dependent from the type of signal reaches the controller that need to elaborate it:

- **ToF, encoder, gyroscope:** these sensors use the I2C interface, a synchronous transmission system that uses two wires: SDA and SCL (serial data line and serial clock line), each sensors has an unique address that the master (Arduino or raspberry) uses to identify each peripheral, for the ToFs since they have all the same address we use the shutdown pin to change them.
- **Encoder:** The encoder needs to use a special pin of Arduino, since we need a fast response we can't use a while loop to check if we receive a signal, so we use interrupt pins to count the steps of the motors.
- **Reflection sensor:** The sensor uses a digital signal that is triggered by setting the potentiometer to a certain threshold and is read by raspberry.
- To let raspberry and arduino communicate we use 2 methods:
 - **Fast communication:** Hardware interrupt pins on the Arduino are utilized exclusively for unidirectional communication from the Raspberry Pi to the microcontroller. To prevent signal floating, a pull-down resistor maintains a stable low state when the Raspberry Pi signal is inactive. This high-speed method is dedicated strictly to signaling system resets and immediate black tile detection
 - **Serial communication:** we use the serial interface, an asynchronous protocol that uses two wires TX and RX and is connected through a usb cable. Raspberry makes a request or gives a command to arduino and it responds accordingly with a custom protocol (further details in section 4.a).
- Raspberry - cameras: These devices also use the serial interface to communicate.

3. Hardware

a. Mechanical Design and Manufacturing

The robot has been designed as a modular and robust system based on a square geometry that improves stability during rotations and accelerations. The main structure is made of PLA, a material chosen for its ease of printing and structural rigidity, while the wheel tires are made of TPU to ensure superior grip on uneven terrain and obstacles.

The robot's architecture is organized into three functional levels that optimize internal space and

facilitate maintenance. The lower level houses the motors and the rescue kit deployment system; the intermediate level contains the batteries and power bank; the upper level houses the PCB and control electronics. The symmetric arrangement of all components ensures balanced behavior during acceleration and rotation, guaranteeing uniform responses to movement corrections.

The locomotion system uses 8 cm diameter wheels equipped with a toothed TPU tread, specifically designed to improve traction on ramps and obstacles. The motors are mounted using a dedicated PLA structure integrated into the chassis, connected through two rods of 10 cm length with a diameter of 0.4 cm. These rods are lubricated to reduce friction and mechanical vibrations. Each motor is also equipped with a dedicated suspension system that reduces the impact of terrain irregularities and improves wheel-ground contact, contributing to stable performance on uneven surfaces.

The rescue kit deployment mechanism is integrated into the lower part of the robot, in proximity to the locomotion system. This central position ensures balanced weight distribution and stable release behavior. The system is designed to be compact and does not interfere with navigation or side sensors, while allowing storage of rescue kits and their controlled release based on commands from the control system. The entire design follows a modular and robust approach that allows structural modifications without requiring a complete redesign. The functional separation into distinct modules enables rapid maintenance and the replacement of individual blocks without disassembling the entire system, thereby facilitating corrections of potential design errors during development iterations. The maximum size constraint of 28 cm diagonal is fully respected thanks to the optimization of internal space through the multi-layer structure.

b. Electronic Design and Manufacturing

SENSORS, ACTUATORS AND PCB:

To know the position of the robot in the maze we used time of flight sensors, with an I2C interface (VL6180x). We employed a total of 6, 2 per side and 1 on the front side and one on the back side, so that we could center the robot in the cells.

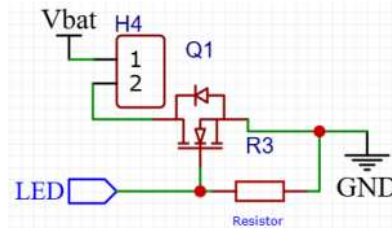
Since ToF sensors operate within a limited close-range window, dead reckoning via a motor-mounted incremental encoder is utilized when forward or backward wall references are unavailable. The software tracks the encoder ticks per rotor revolution to determine precise travel distances. To detect checkpoint tiles (silver cells), a distance sensor calibrated via an onboard potentiometer is employed; it identifies mirrored surfaces and sends a digital signal to the Arduino (low for opaque, high for reflective). Cell color recognition is managed by a *TCS34725FN* sensor, which transmits real-time RGB values to the Arduino via I2C. For orientation, a *BNO055* gyroscope provides absolute pitch, roll, and yaw angles over I2C. The navigation firmware specifically utilizes pitch data to detect ramp inclines and yaw data for closed-loop rotation tracking

Locomotion is driven by four 12V DC motors controlled in pairs directly from the battery supply. To interface the motors with the Arduino, a *TB6612FNG* motor driver is utilized to regulate both direction and power. Since the driver chip features two output channels, the motors on each side are wired in parallel. Control for each channel is managed via three dedicated microcontroller pins: IN1, IN2 (for directional logic), and PWM (for speed regulation).

Input				Output		
IN1	IN2	PWM	STBY	OUT1	OUT2	Mode
H	H	H/L	H	L	L	Short brake
L	H	H	H	L	H	C C W
		L	H	L	L	Short brake
H	L	H	H	H	L	CW
		L	H	L	L	Short brake
L	L	H	H	OFF (High impedance)		Stop
H/L	H/L	H/L	L	OFF (High impedance)		Standby

Reference datasheet

To signal the state of the robot we use a LED strip, that works on 12V, so we use a mosfet to control it:

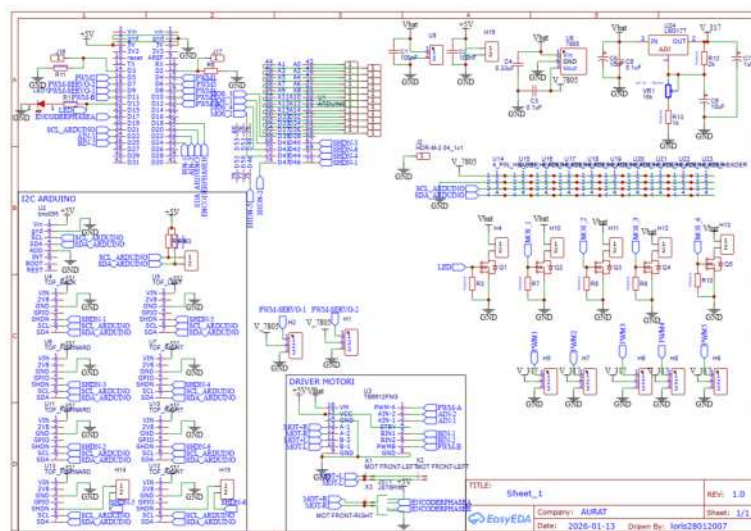


PCB:

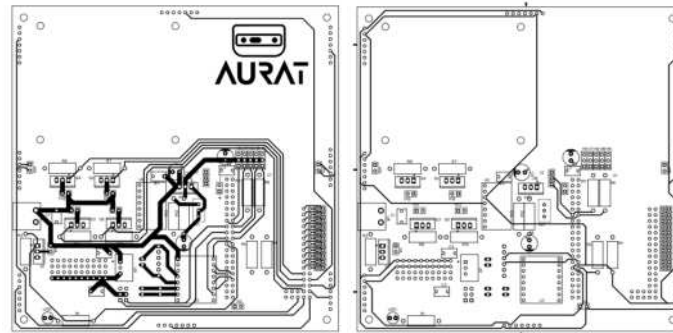
The PCB is custom-designed to route and connect all components in a compact and reliable manner, avoiding the risk of loose wiring or signal interference. Components are strategically placed in their final operational positions directly on the board, significantly minimizing the overall footprint and reducing the reliance on external connectors to only those strictly necessary. The PCB is designed with built-in scalability to accommodate future setup modifications through pre-routed, ready-to-use interfaces. Specifically, the board breaks out 10 additional I2C interfaces, 4 MOSFET channels for high-power outputs, 5 dedicated servo connectors, and multiple uncommitted Arduino digital and analog pins. The system has been upgraded from a single lateral Time-of-Flight (ToF) sensor to a dual-ToF configuration on each side. However, the PCB retains backward compatibility, featuring pre-routed traces and headers to easily revert to the previous single-sensor setup if required. We employ 2 voltage regulators: an *lm317* for the servo power; by so doing, if we need to increase the power of the motors for a specific application we can adjust it, and a *7805* to power the additional power lines for the I2C interface.

For the national competition, a single-layer PCB was manufactured using a CNC mill in the school's electronics laboratory, restricting component placement to one side and causing track insulation issues due to the lack of a protective solder mask. The current iteration utilizes a 4-layer architecture, allocating two internal planes for power distribution (5V and GND) and the remaining two layers for signal routing. This multi-layer design was necessary to accommodate the strict spatial positioning required for the ToF sensors. To simplify routing complexity and ensure maximum connection reliability, the board was manufactured externally by the strategic partner, Futura (responsible exclusively for printing the board, while the design was entirely developed by the team).

The software we used is *EASYEDA*, before we used eagle, but to be sure the exported file would be compatible with the manufacturer we choose it.



Schematic diagram



Top layer

Bottom layer

To check the single parts of the robots we use separate codes for Arduino to be able to identify the problem, to then understand if it's a software or hardware problem.

4. Software

a. General software architecture

Our system uses a distributed, multi-process architecture operating under a Master-Slave paradigm. High-level processes - including navigation, map representation, path planning, and recognition - reside entirely on the Raspberry Pi 5 (Python code). Real-time visual tracking is offloaded to an OpenMV H7 plus executing MicroPython scripts. Low-level hardware execution, real-time sensor polling, and precise motion control are delegated to a dedicated microcontroller (C++ code). This setup makes us able to have two levels of abstraction: the high-level logic in python operates without knowing how raw sensors work (excluding the color sensor), while the microcontroller executes atomic commands without overloading its limited hardware.

Serial communication protocol

In order to make this abstraction work properly, the master needs a way to serialize complex driving decisions into basic execution tokens, which is achieved through a custom string-based communication protocol. Our python system utilizes a serial USB connection at 115200 baud to stream command strings directly to the microcontroller using our custom communication protocol. The microcontroller receives string commands formatted as "x000,", where the "x" represents a character identifier for a specific instruction to execute (e.g., "w" = forward), the "000" represents a three-digit integer value (e.g., "090" = 90), and the comma acts as a message terminator. Within its main loop, the slave continuously polls the serial buffer, Upon detecting a complete string, it processes the data through the following sequence:

1. it reads the leading character identifier to determine the command type;
2. it converts the subsequent three-digit character sequence into an integer value;
3. upon encountering the comma delimiter, it executes the designated hardware command using the parsed parameters.

Maze Mapping & Grid Representation

Every atomic motion token executed corresponds directly to a change in the robot's spatial position. To keep track of these movements and safely explore the environment (e.g., tracking colored tiles, obstacles, and walls), the Python engine translates successful serial movements into coordinate updates within an internal virtual map. The Raspberry Pi models the maze as a 2D grid matrix centered around the starting position of the robot. Each cell represents a 30x30 cm tile and contains a value indicating its exploration status (e.g., 1 = visited, 2 = black tile, 5 = checkpoint). These cells are initialized to zero and updated with real-world classification values once the robot physically occupies the tile. A secondary, synchronized matrix stores wall configurations using 4-bit numbers, where each cardinal direction represents a specific bit weight (North=1, East=2, South=4, West=8). At every step along the path, the master controller processes the Time-of-Flight (ToF) distance values provided by the slave and converts them into an integer representing the surrounding wall positions (for example, walls detected at the North and West generate a combined value of $(1 + 8 = 9)$ at those specific coordinates. This approach allows the robot to rapidly evaluate whether a pathway is open using lightweight bitwise calculations.

A structural challenge within this framework is handling scenarios where the robot drives past the initial boundaries of its memory map. To prevent memory access faults, the software incorporates a dynamic bounds check: if the robot's coordinates approach the edge of the matrix, the system dynamically doubles the grid dimensions, shifts the existing data to the center of the expanded map, and shifts the relative coordinate references accordingly. Consequently, the robot can explore large or highly asymmetrical arenas without experiencing memory overflow errors or losing its spatial orientation.

Concurrent Execution and Multithreading

Maintaining an accurate spatial map requires the robot to handle environmental hazards and structural interruptions instantly. If the main exploration thread had to constantly halt to poll sensors, the robot's reaction times would degrade. So, to guarantee real-time responsiveness, the Python engine handles hazardous events concurrently using a multi-threaded execution layout.

As specified in the rules, the robot cannot cross over a black tile with more than half of its body, so it's crucial to be able to detect one during a movement. Therefore an immediate black tile detection during movement is crucial. We opted for a multithreading architecture, where a secondary thread continuously polls the I2C color sensors in order to immediately capture the presence of a black tile and send an interrupt signal, forcing the microcontroller to instantly invert its drive direction and return to the center of the previous tile before visiting the forbidden zone.

Similarly, in order to properly handle a LoP we've created a dedicated, non-blocking thread that continuously monitors a physical button connected to the Raspberry Pi's GPIO pins. When a LoP is triggered, this thread pauses the active pathfinder, clears all the old movements queues, commands the slave microcontroller to reset its current state, waits out a brief delay for safe repositioning by the captain, and then resets the robot's logical coordinates to the last checkpoint saved in memory. With this approach, we are sure that a LoP event doesn't wipe or corrupt the existing navigation matrices, allowing the robot to seamlessly resume exploration from its new physical starting point.

Graph-based Navigation and Dynamic Pathfinding

The high-level navigation engine models the maze as a dynamic graph tracking visited cells, special tile attributes, and topological constraints. In order to find optimal paths during both maze exploration and the final return-to-home sequence, the software uses Dijkstra's Algorithm paired with a min-priority queue. Rather than treating all tiles as equals, the graph engine assigns discrete, variable motion costs to nodes based on their physical characteristics (explored tile = 2, black tile = infinite cost), balancing traversal efficiency against rule-based time penalties.

The execution loop enforces a preventive look-ahead strategy during path execution. Before moving into a scheduled node, the robot triggers four time-of-flight distance sensors to refresh the local bitmask wall data for all immediate directions. If a newly scanned wall contradicts the planned path layout, the system flags the transition as blocked, cancels the motion sequence, and dynamically re-runs the Dijkstra pipeline from its current coordinates. This runtime calculation loop prevents physical collisions and grid-tracking desynchronization, guaranteeing high reliability under unpredictable maze constraints.

Two-Tiered Computer Vision Pipeline

While parallel threads protect the system from physical hazards and operational resets, the primary goal of this autonomous robot requires dedicated processing resources for victim identification. Our computer vision pipeline underwent a major architectural shift after the national competition: previously we opted for an architecture which relied entirely on the computational power of the Raspberry Pi's CPU executing full-frame deep neural network inference (YOLOv8) to classify letters and targets. It was unreliable, slow and most of all blocking: we could search for victims only when we were completely stopped. Switching to a real-time approach was crucial.

To overcome these embedded constraints, the vision system was bifurcated into a distributed, two-tiered operational framework. Image acquisition and spatial filtering are offloaded to an OpenMV H7 Plus co-processor executing optimized MicroPython.

The first tier executes a dual-stage localization algorithm directly on the edge camera: a wide-spectrum color blob detection isolates potential regions of interest, which are then validated via a localized Hough Circle

Transform to pinpoint precise target boundaries. If a matrix of concentric color profiles is detected within the circle, the OpenMV independently computes the target's validity and color-weighted health score. If an isolated geometric blob lacks target color spaces, the localized bounding box (micro-image) is streamed to the Raspberry Pi 5. This second tier processes only the cropped region of interest using a highly efficient K-Nearest Neighbors (KNN) algorithm. This ensures lightweight, deterministic character classification with minimal memory overhead while the microcontroller executes a controlled hardware pause.

Software architecture and Integration overview

Our robot firmware is designed with the goal of being easy and safe to maintain during all phases of the competition. It's written in C++ for Arduino Mega and divided into well defined functional areas, so we can keep separate all the aspects of the system: sensor, motor, movement control and communication with Raspberry Pi. This organisation allowed us to work on a part of the code without risking introducing problems in others. The structure follows a layered approach. The smallest layer handles directly the hardware: motor, encoder, IMU BNO055, six VL6180X ToF sensors and the color sensor TCS34725. On top of that we find the control layer, which contains the algorithms that run the robot movement: maintaining direction, precise rotation, correction of alignment relative to walls and management of speed ramps. The final layer is dedicated to communication with the Raspberry Pi, which sends commands and receives feedback reporting the movement state.

In order to maintain a stable orientation during the competition, the robot uses quaternions provided by the IMU. This method avoids gimbal lock issues and guarantees a better estimation of the angle. Also, when the robot is required to function in the opposite orientation, as provided by the rulebook, the software remaps the sensors and switches the motor logic, with this the navigation continues functioning without changes.

Problem solving during system integration

During the integration between hardware and software several practical problems have emerged, we solved these problems with direct solutions: the first issue was the robot's slight drift while moving in straight-line motion, even a small difference between the two motors led to a progressive drift. To compensate for this, we introduced a proportional-derivative controller that continuously compares the actual angle with the desired one and adjusts the wheel speeds accordingly. The derivative term was essential to eliminate oscillations and make the movement smoother. The second issue was the alignment of the cells. Even if the robot went straight it might enter in a cell slightly rotated compared to the walls, in order to solve this the firmware calculates the "skew" using the pairs of lateral ToF: comparing the measured distance from the frontal sensor and from the back on the same side, it's possible to obtain the inclination angle. The robot applies a temporary correction before providing and removing it at the end maintaining the global references.

Sensor handling ToF requires particular attention, because all of them share the same I2C address; to avoid conflict, the software activates them one at time with the shutdown pin and gives a different address to all, thanks to this system we can read all of the ToF sensors simultaneously without issues. To comply with the competition rules, the robot must operate even if it's "back to front". Instead of duplicating the code, we implemented an easy flag that inverted the direction of the motors and remapped the sensor, all the navigation logic remained the same and the robot behavior is always predictable.

Tools, Libraries, and Algorithms

The firmware relies on several hardware libraries, including those for the BNO055 IMU, the VL6180X sensor, and the TCS34725. These libraries provide raw data, while the logic developed by the team converts it into important information for navigation.

Among the software components we developed, the most important ones are the motion control algorithms. The straight line controller combines proportional and derivative terms to keep the robot's direction stable. Rotation is handled through a full PID controller, with specific output limits to ensure the robot always has enough force to move even at low speeds. The speed ramping module reduces wheel slip and improves metric accuracy. The skew correction module, based on a simple geometric consideration, allows the robot to automatically align itself with the walls before each forward movement.

Communication with the Raspberry Pi was designed to be lightweight and reliable. Each command is encoded as a single character followed by a numeric value, and the robot responds with a clear status code indicating

whether the movement was completed or interrupted. This allows the high-level software on the Raspberry Pi to react immediately to events on the field.

b. Innovative solutions

Target recognition

The Vision pipeline introduces a spatial-chromatic verification algorithm based on concentric ring profiles, which is fully optimized to execute on the OpenMV H7 Plus integrated processor. The system isolates a candidate blob, then it extracts its radius and center coordinates with a dual-stage pipeline: first, wide-spectrum color blobbing isolates the region of interest, followed by a localized Hough Circle Transform to pinpoint the exact target. Once a target is confirmed, its radius is measured and the full circle gets divided into 5 equal rings, moving from the outside edge to the center. In order to check these 5 zone's color the system places a small rectangular ROI (Region of Interest) over each one.

Within each ROI, an area-voting mechanism is used to calculate the total summation of pixels matching manually calibrated LAB color space thresholds (red, yellow, green, blue, and black). In order to exclude false positives, a strict spatial confidence constraint is applied: a color is validated as the dominant hue of the ring only if its cumulative pixel area exceeds 20% of the total ROI's area. After the dominant color is secured, it gets mapped into its corresponding numerical weight and the overall health status of the target is calculated as the algebraic summation of these weights, all as specified in the competition rules. Once the system determines the final status of the victim, it applies a threshold rule to make an immediate decision. If the final score is negative, the detection is classified as a fake target and is completely ignored. Conversely, if the score is valid, the camera confirms a real victim and forwards the data to the Raspberry Pi 5 to properly handle the rescue routine.

Feature-Based Character Classification via KNN

When the system encounters a blob that doesn't match the properties of a circular target, the bounded region of interest is forwarded to the Raspberry pi for character recognition. In order to classify the greek letters reliably without overloading the CPU, we use a customized K-Nearest Neighbor pipeline built on structural feature extraction rather than computationally neural network tracking.

The custom training structure relies on a balanced local dataset containing 300 sample images per character class. During the preprocessing each candidate frame is binarized using inverted Otsu thresholding and subjected to random data augmentation rotations ranging from 0 to 360 degrees in order to ensure rotation invariance. To eliminate the negative effects of variable line stroke thicknesses, caused by different printing resolutions or camera angles, the binary image undergoes morphological skeletonization. The system then extracts a highly descriptive, low-dimensional 32-feature vector from the skeletonized silhouette. This vector combines global geometric descriptors and spatial density metrics including: Zernike and Hu moments for having a scale rotation invariant shape profile; solidity, circularity, and eccentricity; core pixel density and structural fill ratios. Following this feature extraction, the vector is normalized with the goal of minimizing the influence of extreme data outliers.

The classification is executed using a KNN model configured with $k = 7$ neighbors and Manhattan distance metric, which provides superiors separating boundaries for this feature layout compared to a standard Euclidean distance. In order to prevent distorted, blurry, or partially obscured characters from generating false identifications, which would result in severe point penalties, the system enforces a strict statistical rejection threshold. The absolute distance between the analyzed feature vector and its nearest neighbor in the predicted class is considered to be valid only if it's less than a calculated tolerance: 2.5 times the 95th percentile of the internal distances within the training dataset. If the distance exceeds this boundary, the sample is positioned too far from known class clusters in the vector space and so the software instantly marks the input as a fake target and discards it, successfully suppressing false positives under unpredictable competition conditions.

5. Performance evaluation

Testing Procedures

To guarantee the reliability of our robot under competition conditions, we implemented a structured, two-tier testing protocol focusing on both isolated components (unit testing) and full-arena simulations (system integration testing).

Locomotion and Mechanical Stress Tests: we built a dedicated test bench, featuring a variable-angle ramp and different types of speedbumps (1cm and 2 cm). The robot was tasked with performing many consecutive climbs and descents to evaluate tire grip (TPU tire), chassis stability, and the mechanical endurance of the vertical shock absorbers.

Sensor Calibration and Stop Conditions: we developed a standard multi-cell testing track containing a checkpoint tile, a blue tile, a red tile and a black tile. We performed plenty of iterations to test the coordination between the ToF sensors, the IMU, and the TCS34725 color sensor, verifying if the robot successfully enforced the mandatory 5-second stop without incurring penalties.

Vision Pipeline Benchmarking: during the architectural upgrade after the national competition in Catania, we benchmarked the new two-tiered vision systems. To analyze the true positive and false positive rates of both the OpenMV concentric ring detection and the Raspberry Pi KNN character classifier, we employed a dataset of 300 sample photos per character class under different illumination conditions.

Full Maze Exploration Runs: we simulated the full 8-minute competition on a randomly disposed maze which we previously built (accordingly to the regular maze size, gathered from the rules). Every kind of tile, including dangerous areas, were tested to empirically test our robot's capabilities.

Test Results Analysis and Development Impact

Mechanical & Drive-train: Initial 25° ramp tests showed a low success rate due to wheel slippage paired with poor weight distribution and unstable motor rails. Transitioning to vertical shock absorbers and hollow printed wheel rims increased the success rate to a comfort level.

Vision Architecture: the original full-frame YOLOv8 model running on the Pi CPU was extremely slow and blocking recognition and path following. Offloading blob localization to the OpenMV H7 camera and executing a lightweight KNN classifier on the Pi 5 cut inference time to much lower ms/frame, enabling almost real-time detection.

Note: Other subsystem tests (such as computer vision pipeline benchmarks and serial protocol latency trials) were successfully executed. However, they are omitted from this section to prioritize the analysis of critical physical navigation metrics, as their architectural details and qualitative impacts have been extensively detailed in Section 4.

6. Conclusion

In conclusion, this project allowed us to design a reliable and competition-ready robot that meets all technical, physical, and environmental requirements of the 2026 World Finals. Through a combination of robust firmware architecture, coordinated sensor systems, and innovative mechanical solutions, we achieved stable navigation, accurate victim detection, and consistent performance across all maze conditions. The development process strengthened our teamwork and our ability to solve complex problems under strict constraints. Looking ahead, the experience gained during this season provides a solid foundation for further improvements and more advanced exploration strategies in future competitions.

References

- [1] TB6612FNG datasheet: <https://www.mouser.it/datasheet/3/1466/1/216DCAEB880587735ABECF157C737FDE2A898DFC34EBA4541BB25F243D84D2B0.pdf>
- [2] BNO055 datasheet : <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bno055-ds000.pdf>
- [3] TCS34725 datasheet: <https://octopart.com/datasheet/ams/TCS34727FN>
- [5] Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271
- [5] Cover, T., & Hart, P. (1967). *Nearest neighbor pattern classification*. *IEEE Transactions on Information Theory*, 13(1), 21-27